vasic-digital

Vasic Digital

Hero

AI-native software engineering, built to be trusted.

Anyone can wire an app to an LLM in an afternoon. The hard part — the part that decides whether an AI system is a demo or a dependable product — is everything around the model: the provider abstraction that survives an outage, the orchestration that keeps agents on task, the verification that catches a model bluffing, and the governance that proves the whole thing behaves. That hard part is what Vasic Digital builds. We design and ship AI development systems — the models, agents, orchestration, and infrastructure that turn large language models into dependable software — together with the governance layer that keeps them honest. All of it is anchored by one uncompromising rule: a feature is not “done” when the tests pass; it is done when a real user can actually use it, and there is captured evidence to prove it.

About

Vasic Digital is a focused engineering practice building an interconnected family of AI-development products and reusable modules. Rather than one monolith, the work is organized as a fleet: large product applications on top of dozens of small, independently-tested, decoupled modules — so proven parts are reused across every product instead of rebuilt. The backbone language is **Go**, complemented by **Kotlin / Kotlin Multiplatform**, **TypeScript/React**, **Python**, **Swift**, and **Shell**, chosen per job: Go for high-throughput services and libraries, Kotlin for provisioning tooling and cross-platform mobile, TypeScript for typed frontends, Python for AI/ML glue.

What ties the fleet together is discipline made mechanical rather than aspirational. Every project inherits a shared engineering **Constitution** as a Git submodule — so a rule tightened once

propagates across a 140+-repository fleet — and every capability a product advertises must be backed by an automated, evidence-producing test before it counts as shipped. This is not marketing language layered over the work; it is the operating model the work runs on. The compounding effect is the real advantage: because generic concerns live in decoupled, independently-tested modules, a fix or an improvement lands in one place and lifts every product at once, and each new system is assembled from parts that have already earned their trust.

What we do

AI-based development. We build the substrate for AI systems end to end:

- **Multi-provider LLM access** — a first-party abstraction over 40+ providers (Anthropic/Claude, OpenAI, DeepSeek, Gemini, Mistral, Cohere, Groq, xAI/Grok, Qwen, Perplexity, OpenRouter, Together AI, Replicate, Cerebras, Cloudflare Workers AI, SiliconFlow, and local Ollama as a fallback) behind one interface with retries, circuit breakers, and health checks.
- **Agent orchestration** — headless CLI coding-agent control planes, graph-based agentic workflows, multi-round "AI debate" consensus, and DAG/pipeline runtimes.
- **LLM verification** — a trust layer that scores models with a mandatory comprehension gate ("Do you see my code?") plus latency, streaming, function-calling, vision, and embeddings tests, exporting a verified-only configuration.
- **Retrieval and memory** — RAG, vector databases, embeddings, and fused agent-memory engines (Mem0 + Cognee + Letta) with infinite-context compression.
- **Defensive LLM** — guardrails, PII detection, adversarial red-team fixtures, and input canonicalisation.

The Helix product family. Our flagship line spans the full AI-development lifecycle:

- **HelixTrack** — a free-world alternative to JIRA (the flagship of the Helix-Track line).
- **HelixAgent** — an ensemble LLM service that lets multiple models debate and ships the answer they agree on.
- **HelixCode** — a distributed AI development platform that divides work across SSH-managed workers with checkpoint/rollback.
- **HelixLLM** — one binary, six modes: OpenAI- and Anthropic-compatible inference from laptop to cluster, over HTTP/3.
- **HelixCluster** — a distributed operating system for AI compute, from datacenter GPUs to edge handhelds.
- **LLMProvider / LLMOrchestrator / LLMsVerifier** — the provider abstraction, the agent control plane, and the verification source of truth.

- **HelixMemory, HelixSkills, HelixSpecifier, HelixBuilder, HelixTranslate, HelixTerminator, HelixGitpx, HelixOTA, HelixPlay** — memory, governed skills, spec-driven development, application building, verified translation, zero-trust terminals, federated Git, safe OTA updates, and self-hosted cloud gaming.

Tooling and utilities (vasic-digital utils). Product-grade tools that stand on their own: **Catalogizer** (multi-protocol, encrypted media collection management), **Courses-Creator** (markdown-to-video AI course production), **VisionEngine** (computer-vision + LLM-vision UI perception), **DocProcessor** (documentation-to-feature-map for QA), **Docs Chain** (content-hashed bidirectional doc/DB sync), **Herald** (natural-language multi-channel notifications), **task_bridge** (bidirectional task/board sync), and the **Vasic Digital Reusable Module Suite** — the `digital.vasic.*` "standard library" of infrastructure, AI-primitive, and guardrail modules.

Infrastructure automation (Server Factory). Our DevOps heritage: **Mail Server Factory** and the **Server Factory Core Framework**, which turn declarative JSON into fully-provisioned, Dockerized servers across many connection types and Linux distributions, plus VM-image tooling (Qemu-Utils, Parallels-Utils) and supporting service factories.

Technologies

Grounded in our actual stack:

- **Languages:** Go (dominant), Kotlin & Kotlin Multiplatform, TypeScript, Python, Swift, Shell, with PL/pgSQL and even TLA+ formal specifications in distributed-systems work.
- **AI / LLM:** multi-provider access (43+ adapters), Model Context Protocol (MCP), RAG, vector DBs and embeddings, planning algorithms (HiPlan, MCTS, Tree of Thoughts), LLMOps, benchmarking (SWE-bench/HumanEval/MMLU), and TTS (Bark, SpeechT5).
- **Backend:** Gin (Go), gRPC + Protocol Buffers, HTTP/3 (QUIC), WebSockets, Angular and React frontends, Kafka/RabbitMQ messaging.
- **Data:** PostgreSQL, SQLite, SQLCipher (encrypted at rest), Redis, Neo4j, ClickHouse, and object storage (MinIO/S3/GCS/Azure).
- **Infra / DevOps:** Docker & Compose, Kubernetes + Helm, Prometheus + Grafana, OpenTelemetry, QEMU/Libvirt/Parallels, and CI/CD via GitHub Actions, Gradle, and Make.
- **Testing / QA:** the anti-bluff HelixQA framework, per-module Challenge harnesses with mutation gates, `go test -race`, visual-regression tooling, ADB device testing, SonarQube gates, and

security scanning (semgrep, gosec, trivy, snyk, gitleaks, nancy).

Quality & governance — our differentiator

Two pillars make the whole fleet coherent and trustworthy:

- **HelixConstitution** — a universal, project-agnostic engineering rulebook shipped as a Git submodule and inherited by every project across a 140+-repository fleet. It encodes non-negotiable discipline — anti-bluff evidence gates, false-positive immunity, data and host safety, documentation and coverage rules — that a project may extend but never weaken. One submodule bump upgrades the rules everywhere; propagation gates literally grep for required clauses across the fleet, and every gate is paired with a mutation test that proves the gate itself isn't a sham. Governance becomes an auditable fact, not an aspiration.
- **HelixQA** — anti-bluff QA orchestration. It runs written YAML test banks and fully-autonomous, LLM-plus-computer-vision QA sessions across Android, Android TV, Web, and Desktop, and it refuses to score a PASS without captured runtime evidence (screenshots, logcat, video, stack traces). "We tested it" becomes "here is the video, the logcat, and the ticket."

Positioning statement

Anyone can wire an app to an LLM. Vasic Digital builds the part that is hard: AI systems that are verifiable, reusable, and honest — a provider-agnostic AI substrate, a lifecycle of Helix products on top of it, and a constitution-plus-evidence discipline that guarantees what ships actually works. We don't ask you to trust the green checkmark. We show you the evidence behind it.

Contact

Let's build something verifiable.

- **Email:** i@mvasic.ru
- **GitHub:** github.com/vasic-digital